



Hello! How is it going?



like the dog in the gif
sadlyu i'm sick

sleepy **good**

going

magnificent

потужно

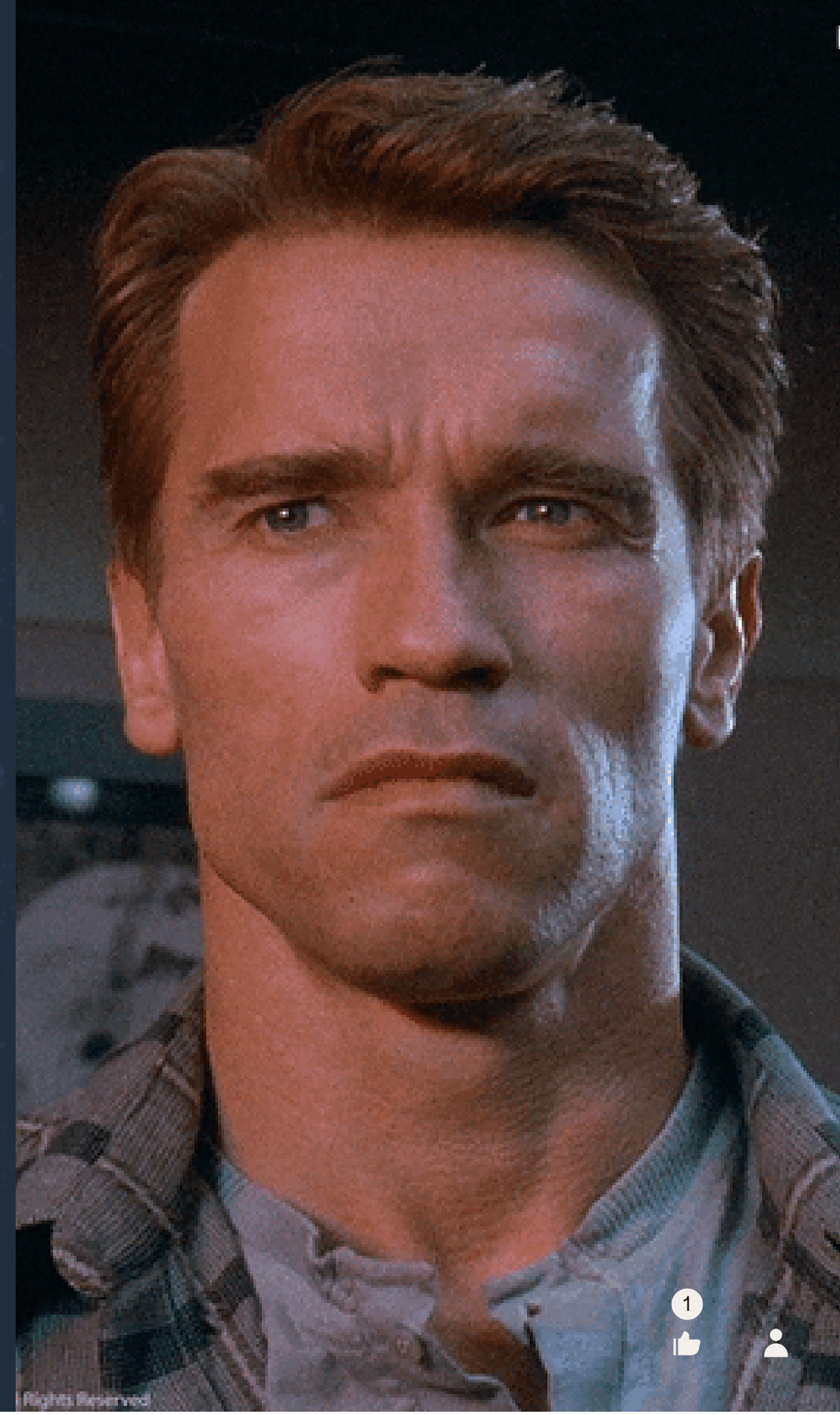
just ok

fine

Agenda

- Recall last session
- Discuss current topic
- Practice
- Quiz

Total Recall



What do you remember most from few previous workshops? (short answer)

java

jdbc

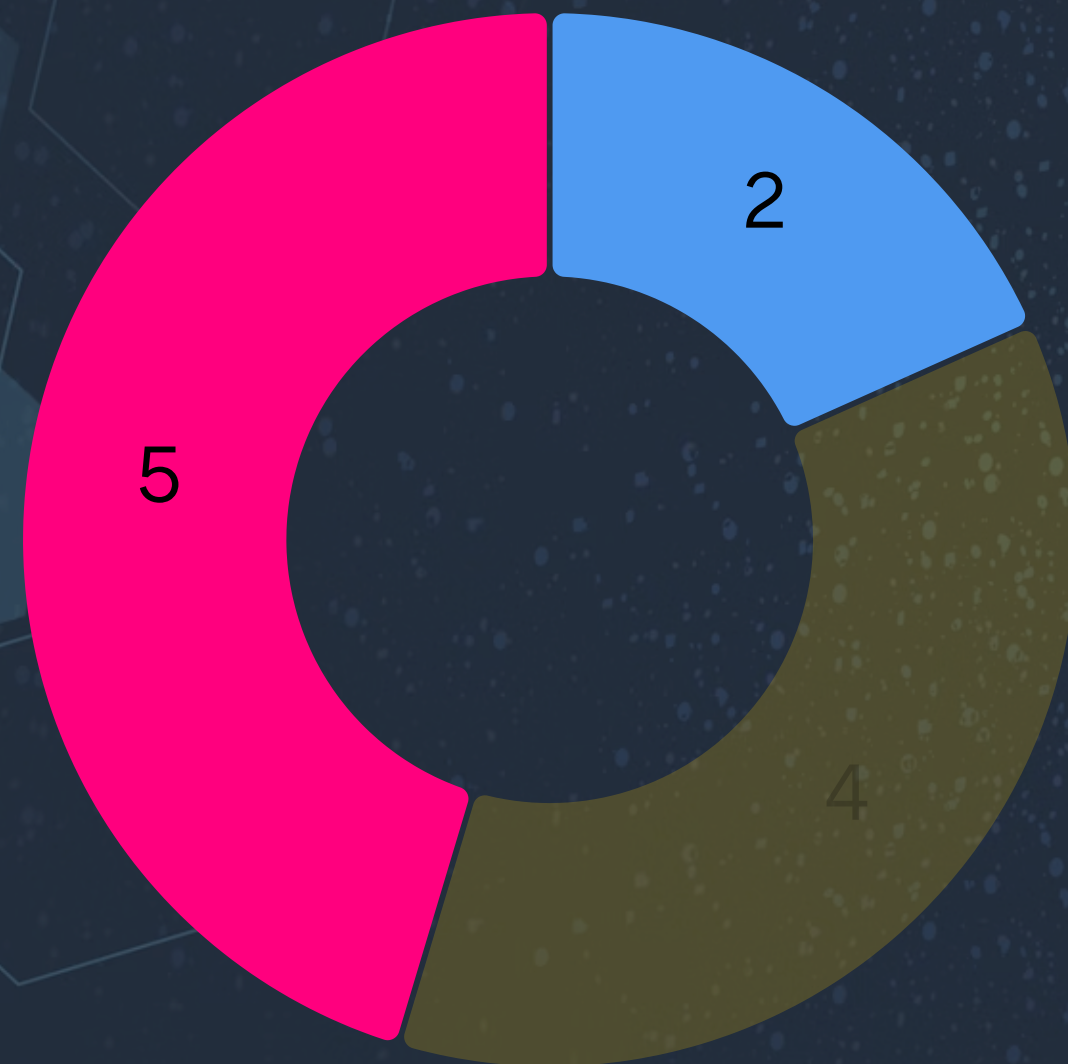
threadpool

Threads

Thread, Runnable and
Callable.

Trump

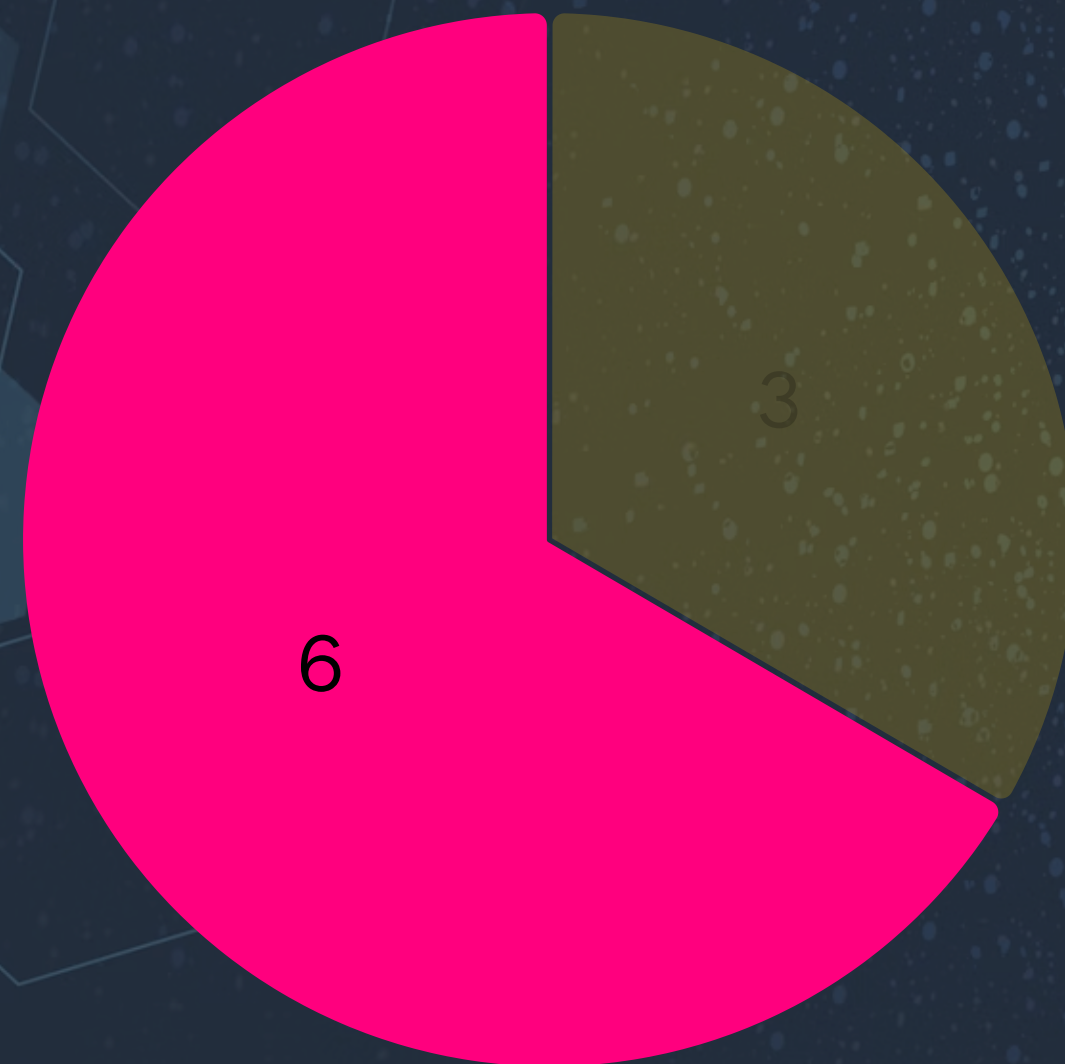
I am typing text with my both hands - is it a concurrent or parallel activit



- 2 concurrent
- 4 parallel
- 5 concurrent-parallel



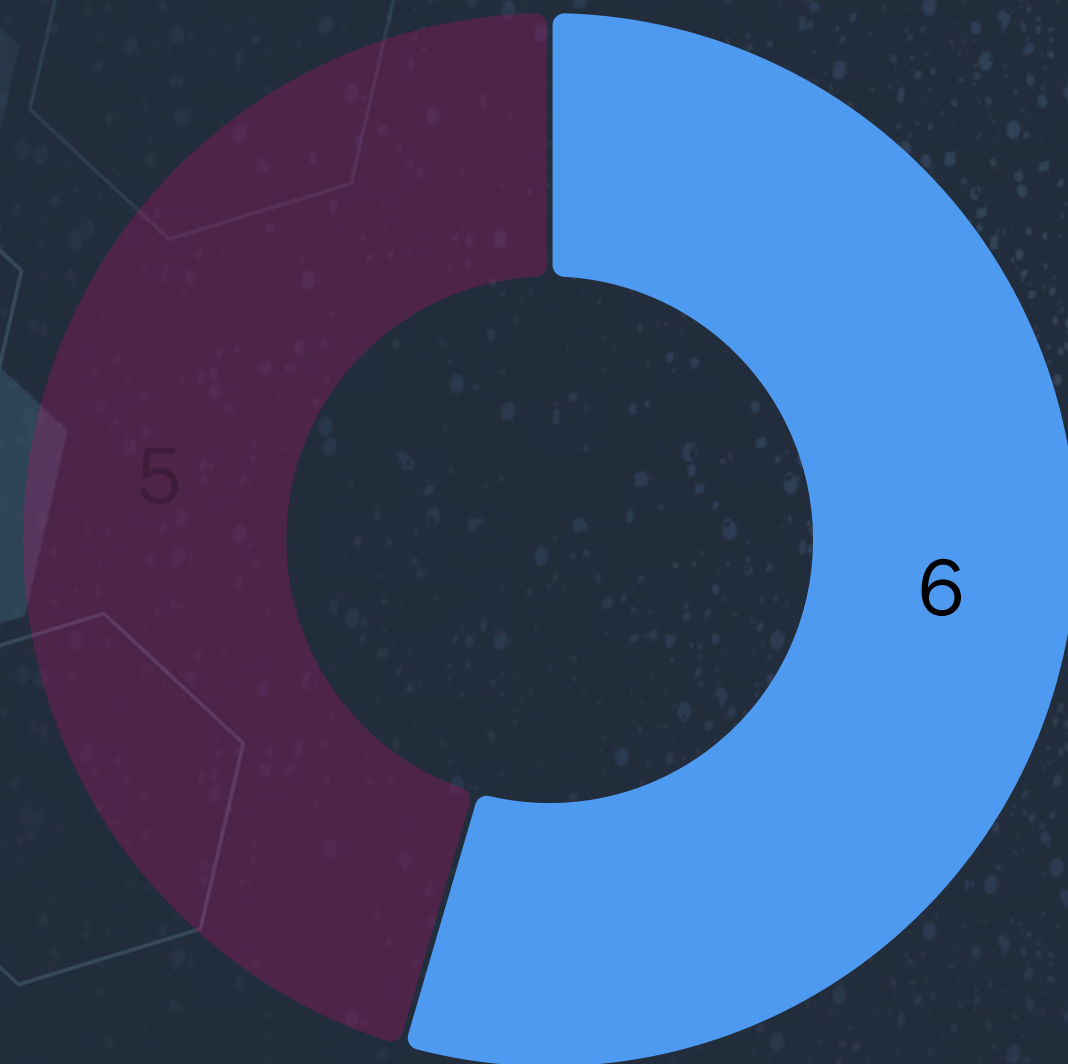
I am walking and listening to the music - is it a concurrent activity or parallel?



3	concurrent
6	parallel
0	concurrent-parallel



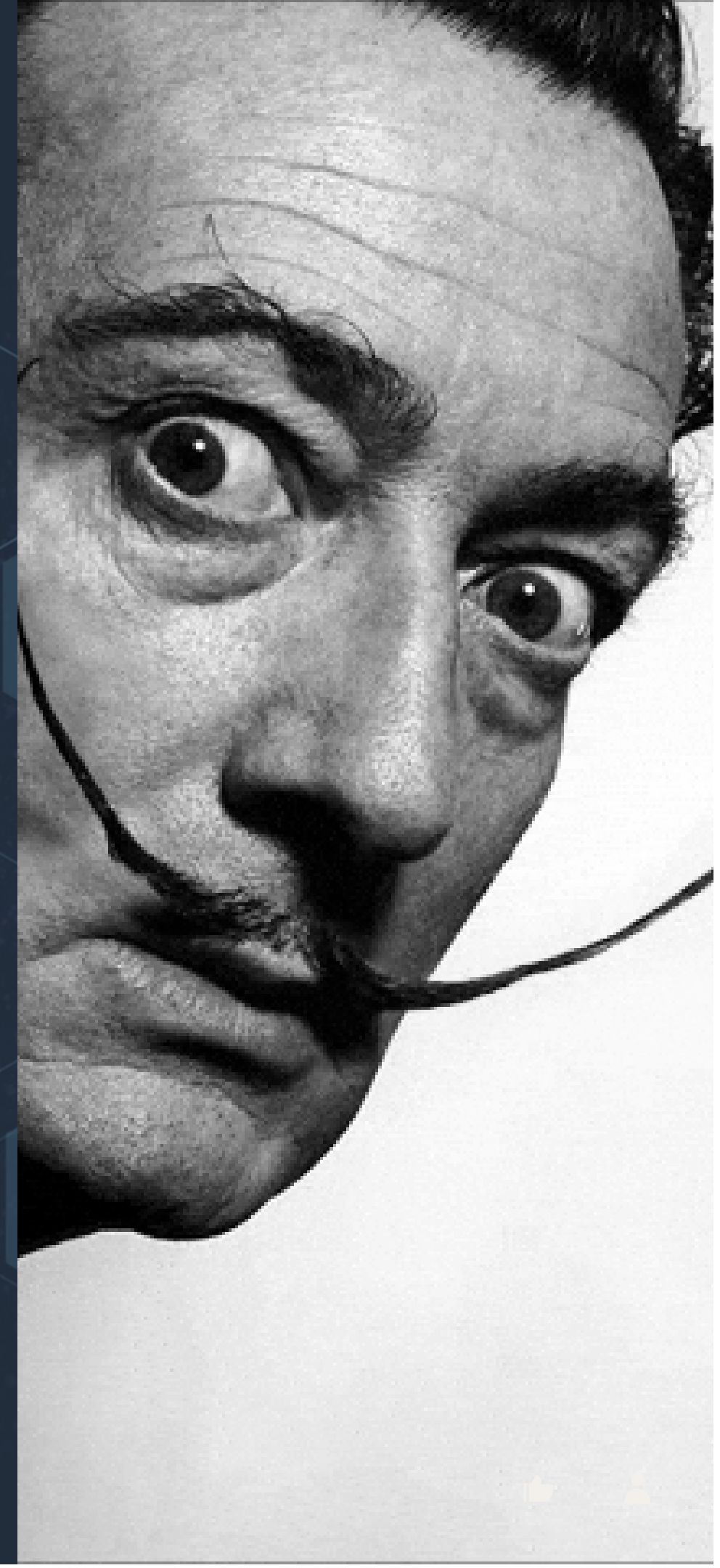
I am eating and drinking - is it a concurrent or parallel activity?



6	concurrent	✓
0	parallel	✗
5	concurrent-parallel	✗

Real world example of concurrency vs parallelizm

Next topic



Random fact

Salvador Dalí designed the logo for the Chupa Chups lollipop brand.



1. Introduction to Synchronization

PROBLEM

Communication between threads may produce two types of errors:

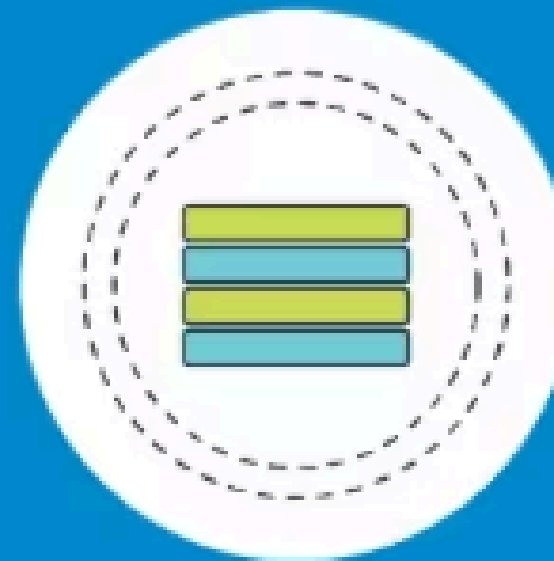
- Thread interference
- Memory consistency

SOLUTION

Synchronization - is the best way to prevent such errors.

Synchronization is a process that allows you to order (divide) access to resources among several threads:
synchronization leads to sequential execution of threads

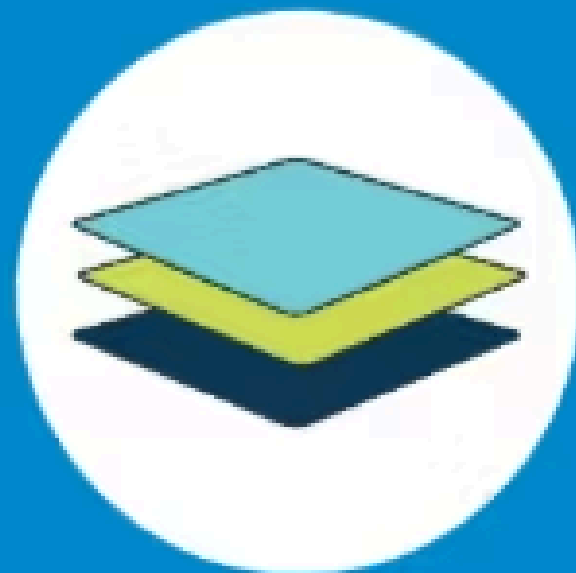
Before a thread can use it, any variable is read from the main memory (heap); any variable written by a thread is written to the main memory.



Stack Memory



The Ambiguity Effect

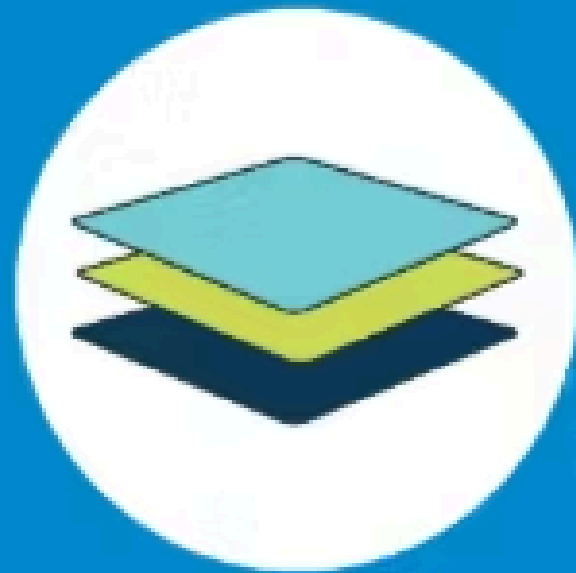


Heap Memory

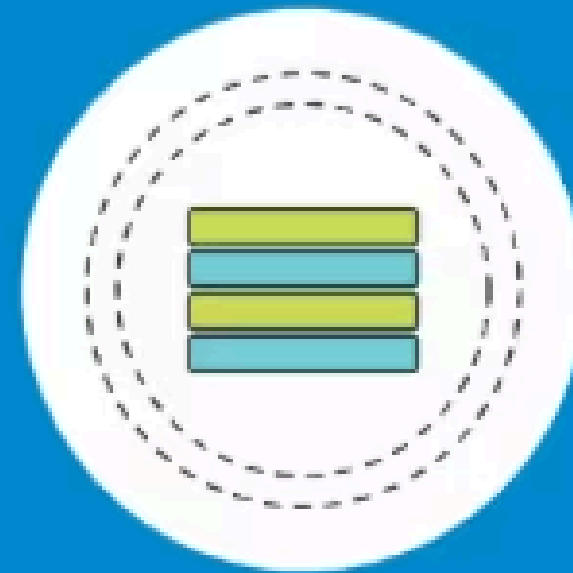
The JRE tries to increase efficiency and saves a local copy of a variable in each thread (stack memory). These copies of variables inside threads help avoid having to check the stack memory when access to a variable value is required.



The Ambiguity Effect



Heap Memory



Stack Memory

The changes made to a copy of a variable inside one thread are not always visible to other threads immediately. The exact times when the JVM reads or writes data from the stack memory to the heap (or vice versa) is unknown.

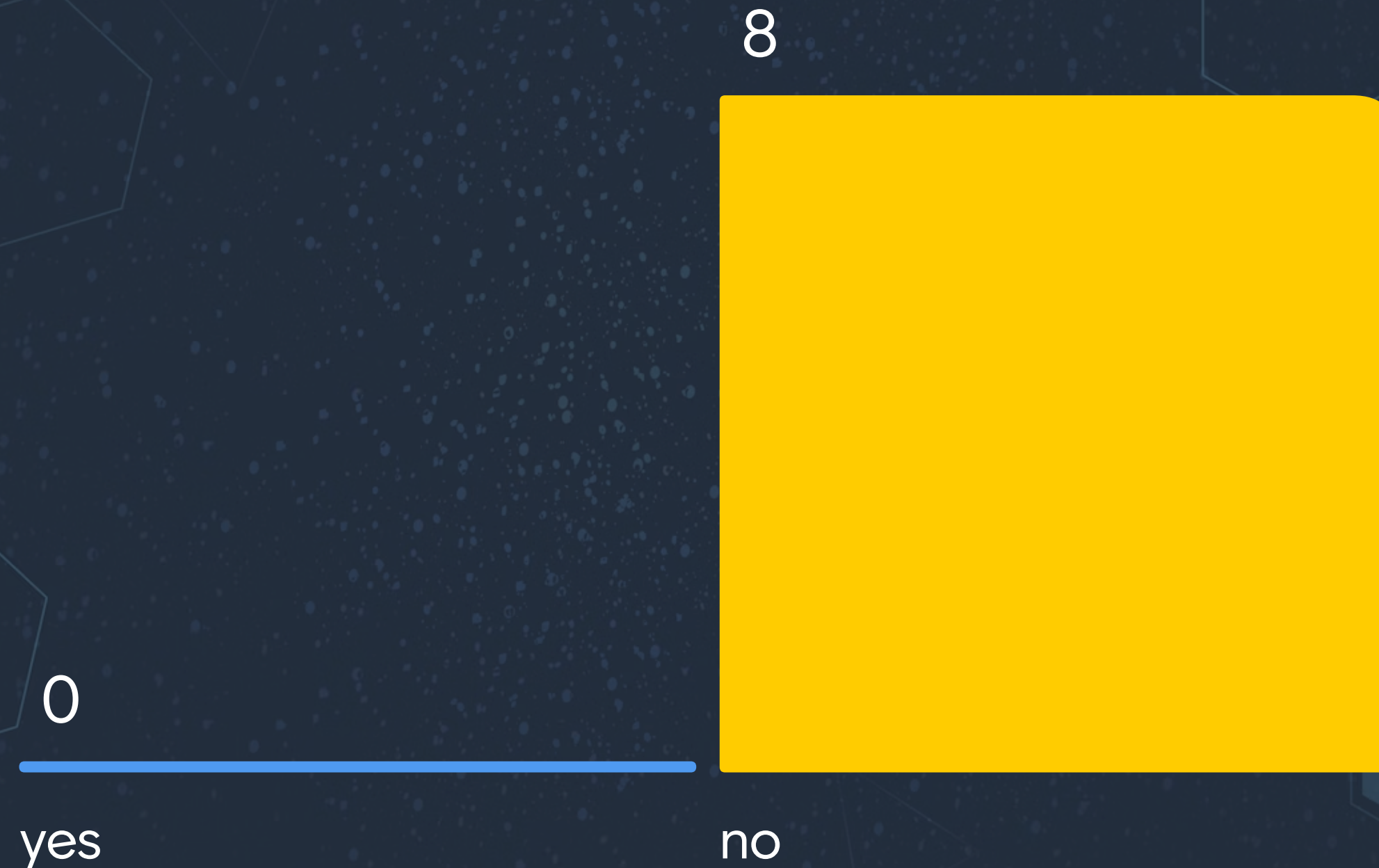
Volatile:

- **Visibility of Changes:** guarantees that changes made to a variable are visible in all threads
- **Access to Any Thread:** `var` is always read from the main memory and never saved in the thread stack, which means changes to it are always available for any thread
- **Request Priority for Writing or Reading:** when several threads request to write and read data at the same time, the system guarantees that the write request is executed first
- **Atomic Operations:** reading and writing operations are guaranteed to be atomic. Other operations such as `++` require blocking synchronization
- **Resource Release:** threads are not blocked while waiting for resources to be released

VolatileDemo



Can we face issues and errors if: we are executing code on 1 core single thread CPU



2. Ways of Synchronizing Threads

2.1 Blocking Synchronization Based on a Monitor

- A monitor is an object used for blocking access to a common resource
- Each object in Java has an implicit monitor, and a thread can use it to lock or unlock an object
- When a thread acquires a lock, it is said that it enters the monitor. The rest of the threads trying to enter a monitor on the same object will be blocked, which is called waiting for the monitor.

VolatileDemoSolution

Different types of synchronization:

- Non-blocking synchronization - volatile or concurrency
- Blocking synchronization
 - Basic synchronization based on synchronized methods and blocks
 - Synchronized Methods
 - Synchronized Blocks
 - Synchronization based on locks (a high-level API from the concurrent.locks package)
 - Synchronization With the API `java.util.concurrent.locks` Package
 - The Lock Interface Methods

Synchronized Methods Workflow

1. Lock Acquisition: thread calls the method and JVM automatically sets a lock on the entire object (enters the monitor).
2. Exclusive Execution: The thread executes the method code. Any other thread (e.g., t2) attempting to call any synchronized method on the same instance must wait until the lock is available.
3. Automatic Release: The lock is released (exits the monitor) once the method finishes—even if it ends due to an exception.

Key Guarantees:

- Mutual Exclusion: Only one thread can execute synchronized code on the same object at a time, preventing race conditions.
- Visibility ("Happens-Before"): Java guarantees that all changes made by the first thread are fully visible to the next thread that acquires the lock.

Coding
pt_2 -> ex2 && ex3

Synchronized Blocks:

Synchronized blocks allow you to protect specific sections of code rather than entire methods

- **Narrowed Scope:** Only the critical code is synchronized, improving performance by allowing non-critical code to run in parallel.
- **External Synchronization:** Useful when you cannot modify a class's source code but need to synchronize on its instances.
- **Flexible Locking:** You can synchronize on any object (the "lock"), providing more granular control than synchronized methods.

The Mechanism

1. **Request:** A thread attempts to enter the block by "grabbing" the specified object's monitor.
2. **Wait:** If another thread holds the lock, subsequent threads are **blocked** in a queue.
3. **Release:** The lock is released immediately upon exiting the block, allowing the next thread in line to proceed.

Coding pt_2 -> ex4 && ex5

BREAK



Advanced Synchronization: `java.util.concurrent.locks`

Why Move Beyond synchronized?

The standard synchronized keyword is simple but limited:

- No Fairness: Threads can wait indefinitely with no guaranteed order.
- Uninterruptible: A waiting thread cannot be cancelled or timed out.
- All-or-Nothing: You cannot "check" if a lock is available without stopping.

The Lock Interface: Key Capabilities

The Lock interface (implemented by ReentrantLock) adds professional-grade control:

- `lock()`: Standard blocking (like `synchronized`).
- `lockInterruptibly()`: Allows a waiting thread to respond to an interrupt signal and stop waiting.
- `tryLock()`: Non-blocking attempt. Returns `false` immediately if the lock is busy.
- `tryLock(timeout)`: Waits for a specific duration before giving up.
- `unlock()`: Manually releases the lock (must be in a `finally` block).

Specialized Lock Classes

- **ReentrantLock** : Allows a thread to re-acquire its own lock and provides "fairness" settings.
- **ReentrantReadWriteLock** : Boosts performance by allowing **multiple readers** simultaneously, but only **one writer**.

Coding pt_2 -> ex6

3. Interthread Communication

Mechanism: Object Methods

Since every object in Java has a monitor, every object provides these three methods:

- `wait()` : Causes the current thread to **release the lock** and sleep until another thread wakes it up.
- `notify()` : Wakes up **one** random thread that is waiting on this specific object's monitor.
- `notifyAll()` : Wakes up **all** threads waiting on this object's monitor.

Critical Requirement: You can only call these methods from within a synchronized context (method or block). If you call them without holding the object's lock, Java will throw an `IllegalMonitorStateException`.

(pt 3 -> ex1 && ex2 && ex3)

1. When a thread calls wait():

- Lock Release: releases monitor (lock) of the current object.
- Suspension: The thread is placed into the Wait Set.
- Efficiency: consumes zero CPU cycles while waiting.

2. When another thread calls notify() or notifyAll():

- Wake Up: The waiting thread is moved from "Waiting" state to "Blocked" state.
- Competition: It does not resume execution immediately. But compete to re-acquire the object's lock.
- Resumption: When captures lock, it moves to the "Runnable" state and continues from the point after wait().

3. Key Requirements

- Must hold the lock: You can only call wait/notify from within a synchronized block/method.
- Implicit Monitor: Accessing these outside a synchronized area results in an `IllegalMonitorStateException`.
- Uncertainty: With notify(), random thread from the WaitSet

The **ReentrantReadWriteLock** is a specialized synchronization tool designed for high-performance scenarios where **reading** happens much more often than **writing**.

Unlike a standard lock that blocks everyone, this class provides two separate locks from a single source:

- Read Lock (R): Multiple threads can hold this lock simultaneously. If the data isn't being changed, any number of threads can read it at the same time.
- Write Lock (W): Only one thread can hold this lock. While a writer is active, all readers and other writers are blocked.

Key Rules of Engagement

- Reader Priority: If several threads want to read, they get access immediately.
- Writer Suspension: If a thread tries to write while others are reading, it is suspended until every single reader has finished and released the lock.
- Mutual Exclusion: No one can read while a write is in progress, ensuring data consistency.

pt_3 -> ex4 && ex5

Deadlocks

A deadlock refers to a situation where two or more threads are locked forever while waiting for each other (e.g., to terminate).

A deadlock is an error that occurs when threads have a circular dependency on a pair of synchronized objects.

pt_3 -> ex6

QUIZ





Quiz time

What does the volatile keyword do?

0 ✗

Saves thread state

1 ✗

Optimizes performance

16 ✓

Ensures visibility of changes
across threads

0 ✗

Locks field in memory

When should volatile be used?

0 ✗

For immutable variables

0 ✗

For thread-local objects

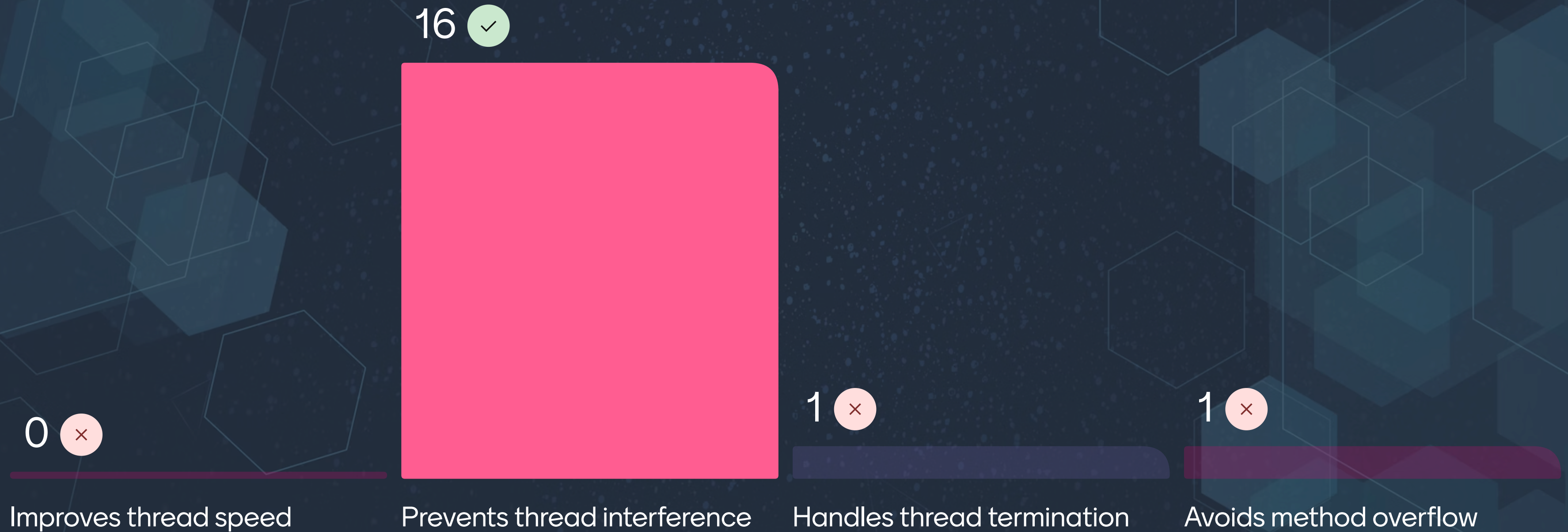
20 ✓

When multiple threads access a variable

0 ✗

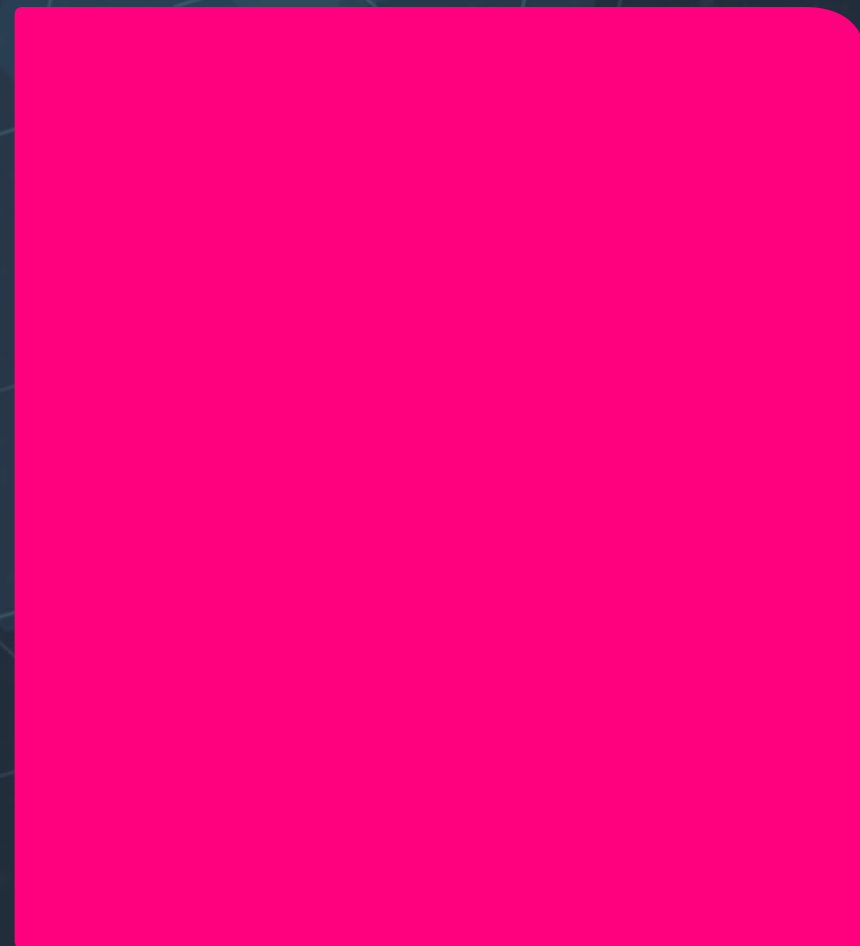
For single-thread operations

What issue does synchronization solve?



How do you apply synchronization?

18 ✓



Use `synchronized` keyword

0 ✗

Override thread joining

1 ✗

Add @Sync annotation

1 ✗

Pass threads through a constructor

What does wait() do in interthread communication?

0 ✗

Creates parallel operations

6 ✗

Pauses thread execution for
duration

14 ✓

Causes thread to wait indefinitely

0 ✗

Returns thread logging

What must a thread acquire to call wait()?

14 ✓



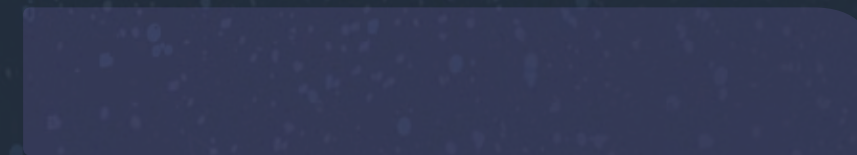
Object monitor

4 ✗



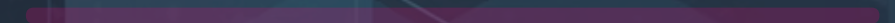
Local variable closure

2 ✗



Thread-final state

0 ✗



Runtime system pointer

How does notify() work?

1 ✕

Terminates thread execution

0 ✕

Waits for thread response

18 ✓

Wakes up a waiting thread

0 ✕

Skips thread execution

How does Thread.sleep() differ from wait()?

5 ✗



Pauses thread permanently

0 ✗

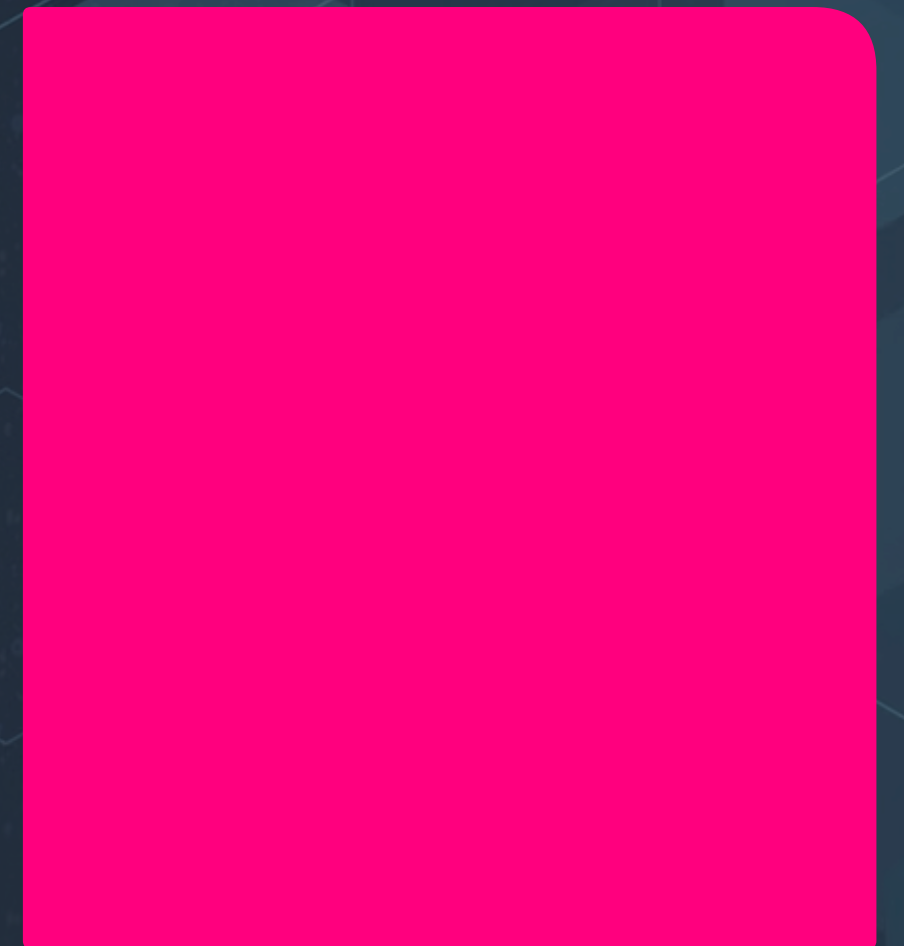
Can wake threads randomly

2 ✗



Requires an object monitor

13 ✓



Pauses thread for specified time

What does synchronized(this) do?

18 ✓



1 ✗

Locks the class instance

1 ✗

Marks the class as immutable

0 ✗

Finalizes runtime tasks

What does Thread.interrupt() signal?

1 ✗

Signals final thread execution

0 ✗

Marks object variable clean

0 ✗

Sends signals across stacks

17 ✓

Request to stop execution of a thread

Quiz leaderboard



Q&A part

1 question
0 upvotes



